GENERATIVE AND AGENTIC AI · TOPIC 8

Agentic Retrieval-Augmented Generation

From Classic RAG to Advanced RAG to Retrieval the Agent Drives

Laura Wynter, PhD
SMU School of Computing and Information Sciences
MITB CS614 · AY2026–27

ROADMAP

What we'll cover

01 What RAG is, and why it helps

02 The simpler alternative: long context

03 The RAG pipeline: chunk → embed → retrieve → generate

04 Embedding models & vector stores

05 Why RAG fails (SAGE)

06 Evaluating a RAG pipeline

07 Re-ranking: the two-phase approach

08 Contextual RAG, BM25 & rank fusion

09 Advanced RAG & the frontier

10 Agentic RAG: retrieval as a tool the agent drives

11 Agentic retrieval patterns & when to use them

12 Evaluating and securing agentic RAG

FRAMING

Learning outcomes

- Understand Retrieval-Augmented Generation (RAG) and how it works end-to-end
- Be aware of its benefits and its limitations
- Understand how those limitations can be addressed and reduced
- Situate RAG in the 2026 stack: modern embeddings, vector DBs, RAGAS, advanced retrieval
- Turn a fixed retrieve-then-generate pipeline into agentic RAG, where the agent (Topic 7) decides what to retrieve

FRAMING

Where we are

- T1–T4: representation, transformers, training and customising the model itself
- T5–T6: serving it, and measuring whether the output is any good
- T7: the agent, an LLM in a loop with tools, memory and guardrails
- Today: the single most useful tool you can give that agent, retrieval over your own data
- We build it in order: what RAG is (§1–§4) → making it accurate (§5–§7) → handing the controls to the agent (§8)

01

What RAG Is, and Why It Helps

Let the model read your data at answer time instead of only its training data.

§1 · DEFINITION

What is RAG?

using external retreiver & not relying on next token predicted in LLM to retreive the information

if you have documents to know where the true answers are then why rely on LLM to predict answer

Augment LLM capability by using external retreiver (SEARCH)

- RAG lets an LLM use user-designated external data sources as context to answer a query
- It combines a "retriever" of information with a "generator" of output
- The retriever fetches data, from the Internet or (usually) a vector database, and hands it, with the original query, to the generator
- The generator uses the retrieved data to produce its response
- Helps mitigate, though not eliminate, hallucination and bias, reducing many LLM ethical issues

§1 · CONTRAST

Bare LLM vs. RAG

Bare LLM call

LLM predict next token base on probability in its LLM

- Answers only from frozen training knowledge
- No access to your private / recent data
- Hallucinates confidently when it doesn't know **0 reason why an output should be correct**
- No citation or provenance for claims

RAG

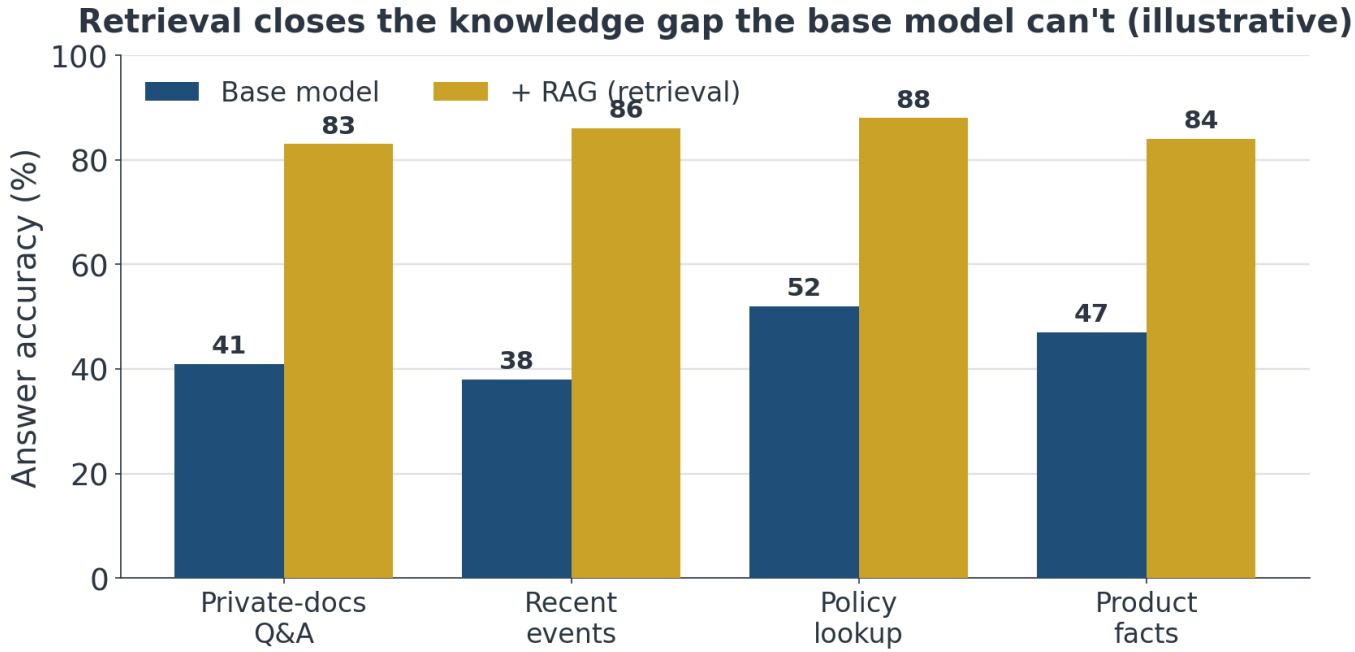
give retreive output to LLM, sources u use can be cited

- Retrieves relevant passages at query time
- Grounds answers in your own corpus
- Fewer hallucinations, answer tied to sources
- Can cite the chunks it used

§1 · WHY IT HELPS

Retrieval closes the knowledge gap

can get higher accuracy when u use external retreiver



Illustrative: on tasks needing private, recent or policy knowledge the base model lacks, grounding lifts answer accuracy sharply.

02

First, the Simpler Alternative

Before building a pipeline, ask whether you even need one.

§2 · LONG CONTEXT

Just put the text in the prompt?

put entire question & document in the prompt

system prompt & user prompt makes no difference just the input & handled in the same way

in a system prompt, it is called anytime a user calls , system prompt always appended to user prompt

input context size for LLM at inference is very huge

- For long-context models (128K+ tokens ≈ 200 pages) you can include the full text directly
- Adding the full text to the prompt can be more effective than RAG, no retrieval to get wrong
- But with a paid API, large input prompts get costly on every call

only do prompt caching if is the information is use everytime

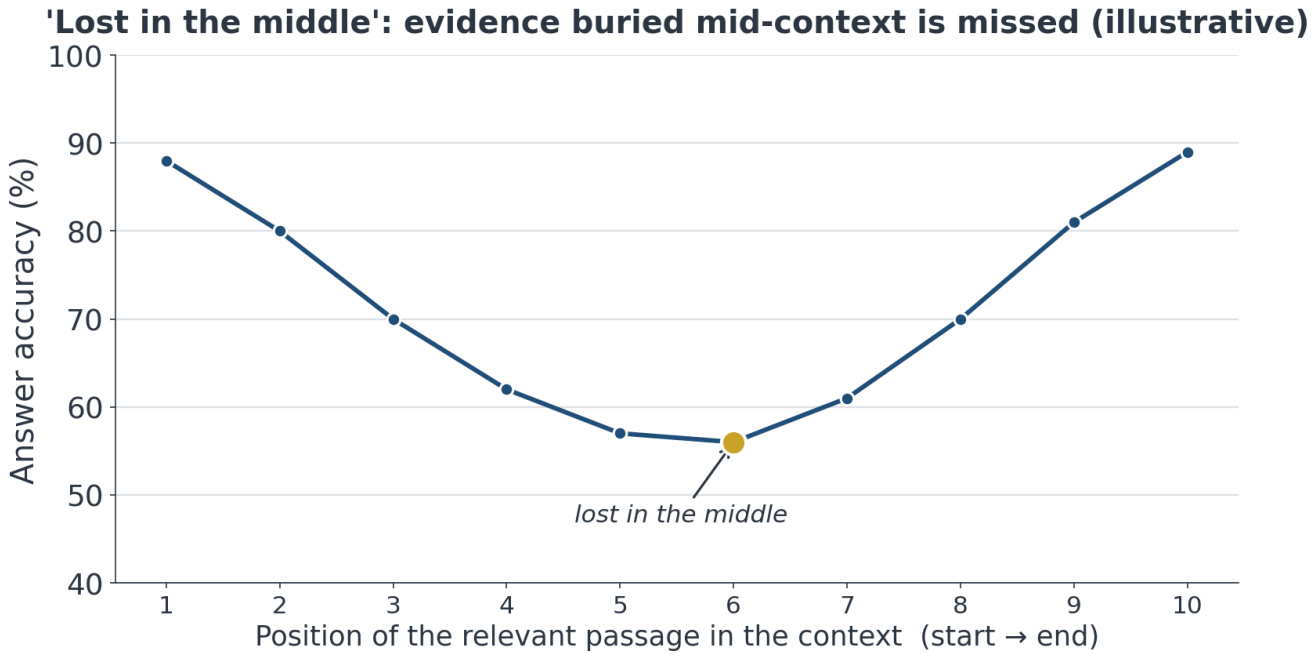
- Prompt caching lets providers reuse a frequently-sent long prompt without re-charging full tokens

KV Cache - if uk system prompt going to us everytime, we already have its kv cache as it is pass everytime —> save on computation

- 2026 reality: context windows now reach ~1M tokens, long-context is a real baseline, not a toy

RAG: baseline whereby u just put ur document in the system prompt and compare how well it works (for model with big enough context window)

Long context isn't free: 'lost in the middle'



Illustrative: accuracy is highest when the key passage sits near the start or end of the context and sags in the middle, a reason to retrieve and surface the right passage rather than bury it in a huge prompt.

§2 · WHEN RAG WINS

For longer material, back to RAG

- Corpora bigger than any window, changing constantly, or needing per-user scoping still favour RAG
- RAG retrieves only what's relevant, cheaper per query than stuffing everything every time
- Rule of thumb: small/static context → long-context prompt; large/dynamic corpus → RAG

§2 · PIPELINE

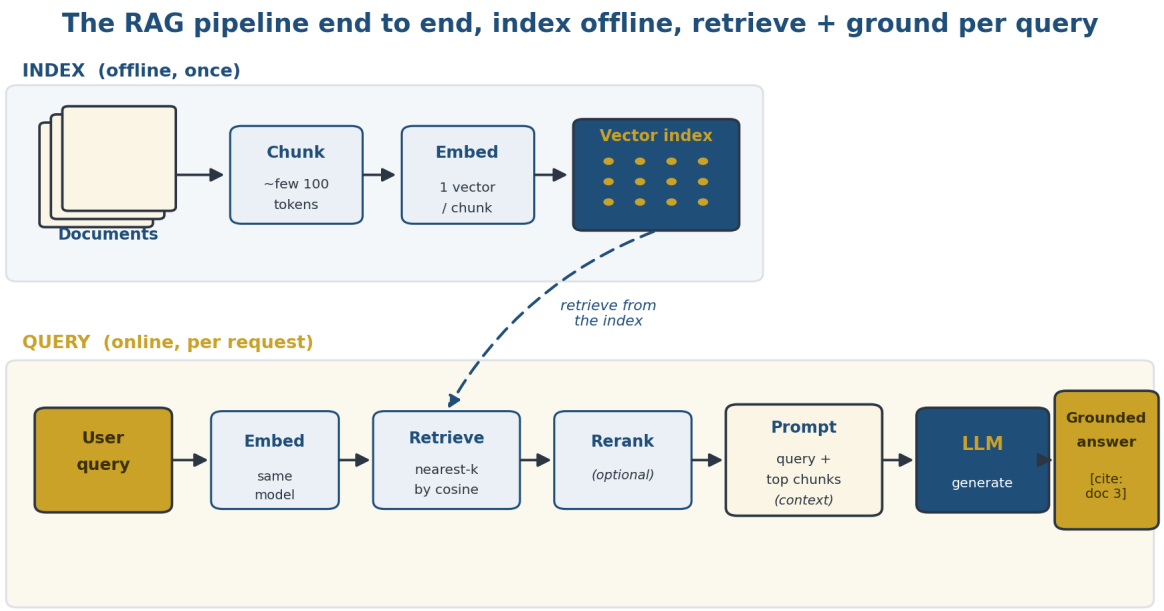
The RAG pipeline, end to end



Index once (chunk → embed → store); then retrieve + generate per query. The same embedding model must encode both the chunks and the query.

§2 · PIPELINE DRAWN OUT

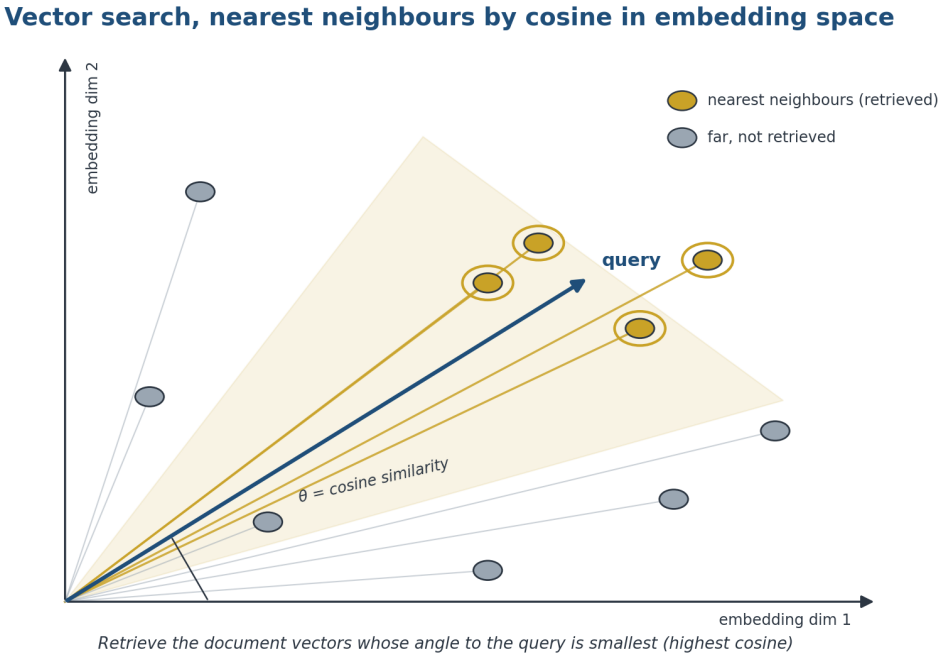
RAG end to end, an offline index lane and an online query lane



Illustrative: index once (chunk → embed → vector index); per query, embed → nearest-neighbour retrieve → optional re-rank → prompt+context → LLM → a grounded, cited answer.

§2 · VECTOR SEARCH

What 'retrieve top-k by cosine' actually means



Illustrative: query and document chunks are points in embedding space; retrieval returns the vectors whose angle to the query is smallest (highest cosine similarity).

§2 · EMBEDDINGS

Embedding models: beyond all-MiniLM

Model	Where it runs	Note
all-MiniLM	Local (ST)	Small, fast, our labs
Gemini Embedding	API (Google)	Current MTEB leader
Qwen3-Embedding	Open weights	Current MTEB leader, self-hostable
BGE-M3 (BAAI)	Open weights	Multilingual, multi-granularity, hybrid-friendly
Cohere embed-v4	API	Multimodal, long-doc, strong retrieval
voyage-3.5 (Voyage)	API	High hosted quality
text-embedding-3	API (OpenAI)	Strong, no local GPU

Pick by: retrieval quality (MTEB leaderboard, leaders shift often), embedding dimension / cost, and local vs. API constraints.

§2 · VECTOR STORES

Storing & searching the vectors

- Our labs use brute-force cosine over all chunks, perfectly fine at teaching scale
- In production, a vector index gives approximate-nearest-neighbour search that scales to millions
- FAISS (in-process library), Chroma (lightweight local DB), pgvector (Postgres extension)
- Managed options (Pinecone, Weaviate, Qdrant) add hosting, filtering and hybrid search

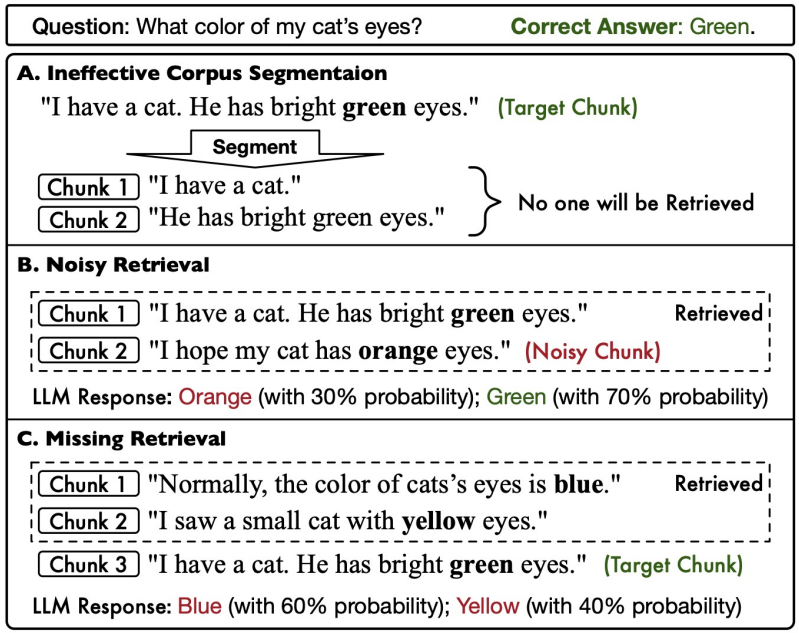
03

Why RAG Fails

RAG works most of the time, engineering is mostly about the times it doesn't.

§3 · THE PROBLEM

How RAG goes wrong, one question, three failures



Question: what colour are my cat's eyes? (A) chunking splits the answer so neither piece is retrieved; (B) a noisy chunk competes with the right one; (C) the target chunk is never retrieved at all. Source: SAGE, Zhang et al., arXiv 2025.

§3 · FAILURE MODES

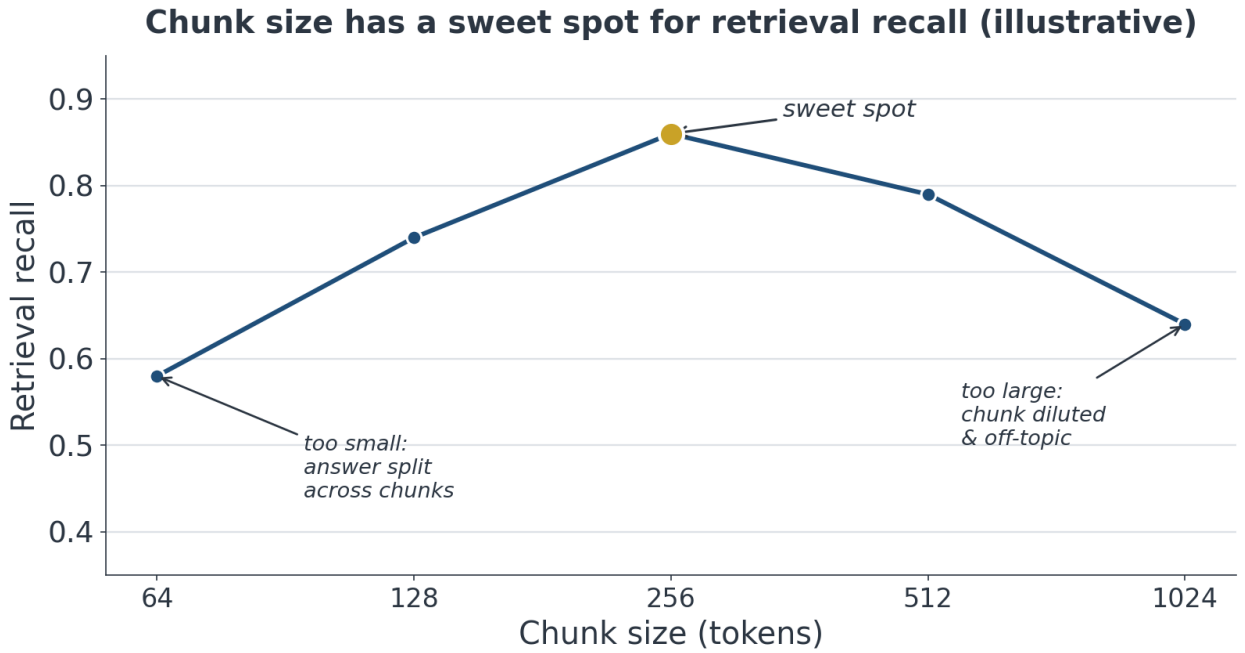
Three ways RAG breaks

Failure mode	Symptom	Fix
Bad chunking	No chunk carries the full answer	LLM / learned chunker (SAGE)
Noisy retrieval	Irrelevant chunks in the top-N	Re-rank; tighten k
Missing retrieval	Target chunk falls outside top-N	Raise N, then hybrid + re-rank

Chunking remedy: ask an LLM to chunk; SAGE trains a dedicated chunking model (next slide).

§3 · CHUNKING

Chunk size is a trade-off with a sweet spot



Illustrative: tiny chunks split the answer across many pieces; huge chunks dilute each vector with off-topic text. Recall peaks at a mid size, tune it, don't guess.

SAGE: precise retrieval for RAG

> *"Precise retrieval, a learned chunker plus semantic segmentation, sharply reduces the noisy- and missing-retrieval failures of naive top-N RAG."*
>
> SAGE: A Framework of Precise Retrieval for RAG · Zhang et al., arXiv 2025

04

Evaluating a RAG Pipeline

To fix ineffective RAG, first measure it, quantitatively.

§4 · THE PROBLEM

Why the top-N prompt can go wrong

- The basic paradigm sends the top-N chunks plus the query straight to the LLM, three things can break it:
  - Chunks add irrelevant info that then leaks into the answer
  - The embedding distance doesn't produce an accurate ranking
  - The N chunks miss the relevant information entirely
- So: quantitatively evaluate the pipeline before trying to improve it

§4 · CLASSIC METRICS

Retrieval metrics with ground truth

- First gather test data with ground-truth answers (and the passages that should be retrieved)
- Score retrieval as a classic IR problem: Precision, Recall, F1 over the retrieved chunks
- Generation overlap can be checked with BLEU and ROUGE against reference answers
- Limitation: BLEU/ROUGE reward surface word overlap, not whether the answer is faithful or correct

§4 · RAGAS

ADVANCED

Modern RAG evaluation: RAGAS

Metric	What it checks
Faithfulness	Every claim in the answer is supported by the retrieved context
Answer relevance	The answer actually addresses the user's question
Context precision	The retrieved chunks were relevant, not noise
Context recall	The retrieved chunks covered the full answer

LLM-graded, RAG-specific metrics, prefer over BLEU/ROUGE. See also TruLens, LangSmith eval.

05

Re-Ranking: The Two-Phase Approach

Recall more, then re-order with a model that actually reads the query.

§5 · THE IDEA

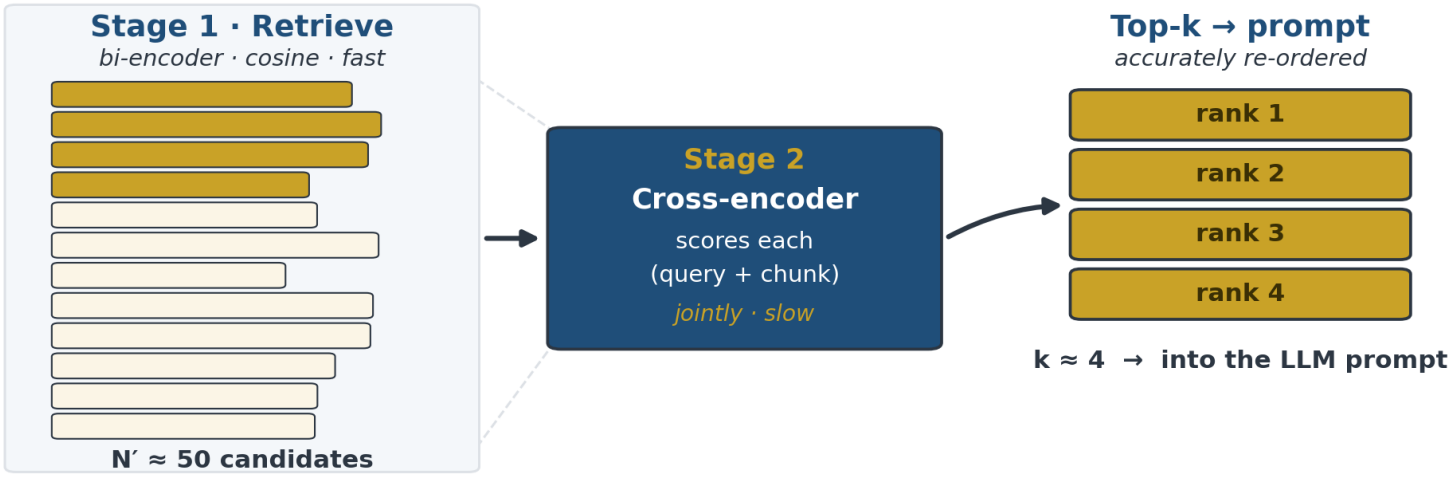
Two-phase RAG with a re-ranker

- If accuracy is poor, re-ranking is the first lever to pull
- Phase 1, recall widely: increase N → N’ so more candidate chunks are retrieved
- Phase 2, re-order: pass the N’ chunks to a "re-ranker" that re-scores and re-orders them
- The re-ranker uses an LLM or specially-trained encoder to score each document with the query as context
- Other backends (e.g. Elasticsearch, which also indexes CSV-like data) can replace the vector DB

§5 · THE FUNNEL

Retrieve wide, then narrow with a cross-encoder

Two-stage retrieve-then-rerank, wide recall, then a precise cross-encoder

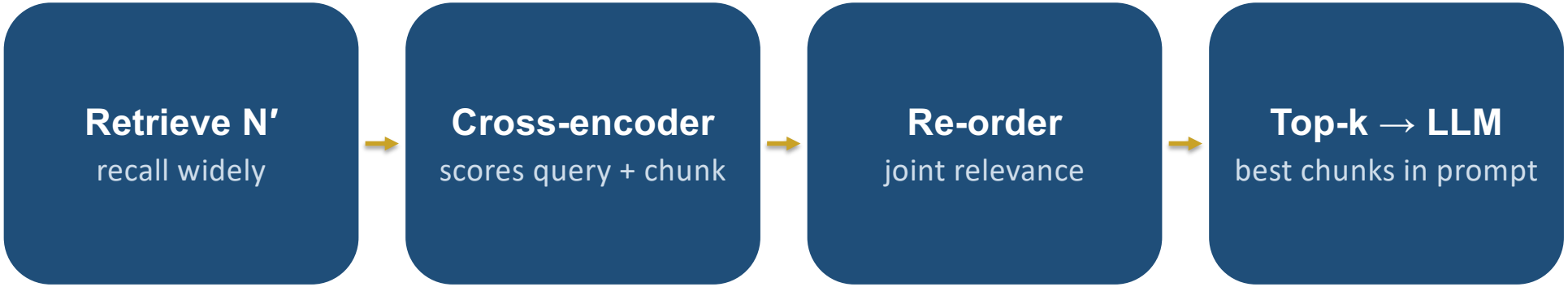


Recall widely and cheaply, then re-order the few finalists with a model that actually reads the query

Illustrative: a large, cheap candidate set from the bi-encoder is re-scored by a cross-encoder that reads the query with each chunk, leaving a small, accurately-ordered top-k for the prompt.

§5 · TWO-PHASE

Two-phase retrieval with a re-ranker



Recap of the funnel above: phase 1 recalls many candidates cheaply, phase 2 re-orders them with a model that reads the query.

§5 · WHY IT HELPS

Why re-ranking beats plain cosine

- The vector DB embedded each chunk when it was loaded, before any query existed
- Those chunk embeddings do not take the user query into account, no Attention over the pair
- A re-ranker feeds the chunk AND the query to a transformer together
- It computes a joint, context-dependent similarity score, a far more relevant ranking

ADVANCED

Bi-encoder vs. cross-encoder

Bi-encoder (the vector DB)

- Chunk and query embedded separately
- Embeddings precomputed, query-independent
- Similarity = cosine of two fixed vectors
- Very fast; scales to millions of chunks

Cross-encoder (the re-ranker)

- Chunk + query fed in together (Sent A + Sent B)
- Joint attention over the pair at query time
- Outputs one context-dependent relevance score
- Much slower, score every candidate at query time

§5 · IN PRACTICE

ADVANCED

Re-rankers in practice

- BERT-type cross-encoders do the re-ranking, also sold as APIs
- Cohere Rerank 4: rerank-v4.0-fast and rerank-v4.0-pro
- Rerank v3.5 is deprecated; since Aug 2026 calls go to rerank-4
- Rerank 4 scores differ from 3.5, re-tune any hard cut-offs
- Advantage: more relevant chunks, better ordering into the prompt
- Disadvantage: far slower than cosine, so re-rank few, not many

06

Contextual RAG, BM25 & Rank Fusion

Anthropic's 2024 recipe: fix lost context and bring back keyword search.

§6 · BM25

Keyword search is not dead: BM25

- Contextual RAG (Anthropic, 2024) resurrected BM25 alongside it
- BM25 extends TF-IDF by also accounting for document length
- A query like "Error code TS-999": embeddings miss the exact string, BM25 nails it
- Lesson: semantic and lexical retrieval fail differently, combine them
- Combine with Reciprocal Rank Fusion (RRF): sum 1/(k+rank) across both lists

§6 · CONTEXTUAL

Contextual RAG: prepend context to chunks

- Problem: a chunk pulled out of its document loses the context needed to interpret it
- Fix: prepend a short context blurb to each chunk before embedding it
- Give the full text (or main component) plus one chunk to an LLM; get 50–100 tokens of chunk context
- Repeat per chunk, prepend the context, then embed, retrieval now sees each chunk in situ

§6 · PIPELINE

Contextual + BM25 + rank-fusion pipeline



Anthropic (2024): semantic and lexical retrieval fail differently, fuse the two ranked lists with Reciprocal Rank Fusion (RRF), then optionally re-rank.

07

Advanced RAG & the Frontier

Where retrieval research has gone since the basic pipeline.

§7 · SURVEY

Advanced retrieval techniques

- Hybrid search: run semantic + BM25 together and fuse, now the default strong baseline
- Query rewriting: expand or clarify the user's query before retrieval (multi-query, step-back)
- HyDE: have the LLM draft a hypothetical answer, embed that, and retrieve against it
- GraphRAG: build a knowledge graph over the corpus so retrieval can traverse entities and relations
- Each targets a specific failure: vocabulary mismatch, under-specified queries, multi-hop questions

§7 · VISUAL RAG

ADVANCED

ColPali: visual document RAG

- Problem: chunk-and-embed pipelines run on extracted text, they lose figures, tables and chart layout
- ColPali retrieves over rendered PAGE IMAGES directly, no OCR, no brittle text-extraction step
- It uses late-interaction MULTI-VECTOR retrieval (ColBERT-style): many vectors per page, matched token-to-patch
- Strong on figure-, table- and chart-heavy PDFs, exactly the material in the PDF RAG lab (Lab B)
- Cost: multi-vector indexes blow up storage; MUVERA compresses them into single fixed-size vectors to fix that

§7 · CHOOSING

What to reach for, when

Technique	Reach for it when
Chunking + embeddings + top-k cosine	Always, the strong default baseline
Hybrid (BM25 + semantic)	Exact terms, codes or names matter
Re-ranking	Recall is fine but ordering is noisy
HyDE / query rewrite / GraphRAG	Evaluation shows a specific gap

Each technique targets a specific failure, add them in response to evaluation, not by default.

§7 · OUR LABS

Adding the "G": generation, and what it needs

- Everything so far produced a ranked list of chunks; the "G" turns the top ones into an answer
- Generation is one LLM call: system prompt with a grounding rule + the retrieved chunks + the question
- Grounding rule: answer only from the chunks, cite them by number, say "not in the documents" otherwise
- Both labs end with this step, and you change two dials: the grounding rule, and how many chunks go in
- Watch for the two failures: a wrong chunk answered confidently, and the LLM ignoring the rule and answering from memory

08

Agentic RAG: Retrieval the Agent Drives

Stop hard-coding the retrieval; make it a tool the agent decides how to use.

§8 · THE SHIFT

What changes when the agent owns retrieval

- Everything so far is a FIXED pipeline: one query → one retrieval → one generation, decided by your code
- Agentic RAG makes retrieval a TOOL (Topic 7): the agent decides whether to retrieve, what to ask for, and when to stop
- The loop can search several times, rewrite the query between attempts, and check the evidence it got back
- Nothing from §1–§7 is thrown away: chunking, embeddings, hybrid search and re-ranking are the tools it calls
- What moves is the control flow, out of your script and into the agent's loop

§8 · MEMORY

Retrieval as the agent's long-term memory

- Topic 7: short-term memory only, the message history and a one-task scratchpad
- Long-term memory: embed the agent's own notes, store them, retrieve by similarity
- Each step: embed the question, pull the top-k memories, put them in the prompt
- Same index, embedding model and cosine ranking; only the corpus author changed
- Same failure: similarity is not relevance, a look-alike memory can outrank the right one
- Same fixes: re-rank the hits, ground the answer in them, expire stale memories

§8 · CONTRAST

Classic RAG vs. agentic RAG

Classic RAG (fixed pipeline)

- Always retrieves, exactly once
- Query is used as the user typed it
- Top-k is whatever the index returned
- No recovery if the right chunk is missed
- Cheap, fast, predictable, easy to debug

Agentic RAG (agent-driven)

- Retrieves only when the question needs it
- Rewrites and decomposes the query first
- Judges the chunks, and can search again
- Recovers: widen, re-query, or switch source
- Slower and dearer, better on hard questions

§8 · THE LOOP

The agentic retrieval loop



The same retrieve-and-generate parts as §2, but the arrows are decided at run time by the agent, and "Judge" can send it back to "Search" several times before it answers.

§8 · PATTERNS

Agentic retrieval patterns

- Query rewriting: turn a vague question into a good search query before retrieving (§7, now done by the agent)
- Decomposition: split a multi-part question into sub-queries and retrieve for each
- Iterative / multi-hop: use the answer to sub-question 1 to form the query for sub-question 2
- Self-correction (CRAG, Self-RAG): grade the retrieved chunks; if weak, re-query, widen k, or fall back to web search
- Routing: pick the right source per question, vector store, BM25, SQL table, or the live web

§8 · CHOOSING

Which agentic pattern fixes which failure

Failure (from §3)	Agentic remedy
Target chunk falls outside top-N	Re-query and iterate until the evidence covers the question
Noisy chunks compete with the right one	Grade the chunks, drop the weak ones, retrieve again
Question spans several documents	Decompose into sub-queries, retrieve each, then synthesise
Right answer lives in another system	Route to the tool that holds it (SQL, web, another index)
Question needs no retrieval at all	Let the agent skip retrieval instead of forcing it

Add an agentic step for a failure your evaluation actually shows, the same discipline as the advanced-RAG table in §7.

§8 · DEEP RESEARCH

ADVANCED

Deep research: agentic RAG at long horizon

- “Deep research” products are agentic RAG run for minutes, not seconds
- The agent plans a set of questions, searches repeatedly, keeps notes in memory (Topic 7), then writes a cited report
- Sub-agents can each own one sub-question and report back to a lead agent
- The hard parts are not retrieval: budget control, avoiding loops with no progress, and honest citation

§8 · EVALUATION

Evaluating agentic RAG

- Everything from §4 still applies to the FINAL answer: faithfulness, answer relevance, context precision/recall (RAGAS)
- Add the trajectory metrics from Topic 7: how many retrieval calls, did each one add new evidence, tool-call success rate
- Watch for the characteristic failure: the agent loops, searching over and over without making progress
- Always keep the fixed pipeline as a baseline, agentic RAG has to earn its extra cost and latency

§8 · COST & RISK

What agentic RAG costs you

- Every extra turn of the loop is another retrieval plus another LLM call, slower and dearer per answer
- Cap it: a maximum number of retrieval calls, a token or time budget, and a clear stop condition
- Retrieved text is UNTRUSTED input: an injected instruction can ride in on a chunk and hijack a tool-using agent
- So the guardrails from Topic 7 apply here: scoped tools, no privileged action taken on retrieved instructions, log the trajectory
- Rule of thumb: start with the fixed pipeline, go agentic when evaluation shows single-shot retrieval is what's failing

Key takeaways

01 **RAG = retriever + generator**
Ground the LLM in your data at query time, chunk, embed, retrieve top-k by cosine, then generate.

02 **Know the simpler baseline**
Long-context prompting (with caching) can beat RAG for small, static corpora, measure before building.

03 **Measure, then improve**
Evaluate with Precision/Recall/F1 and prefer RAGAS (faithfulness, relevance) over BLEU/ROUGE.

04 **The upgrade path**
Re-ranking, hybrid BM25 + contextual RAG, then advanced retrieval, each fixes a specific failure.

05 **Agentic RAG**
Make retrieval a tool the agent drives, rewrite, retrieve, judge, retrieve again, and pay for it only when the questions are hard.

LAB

Hands-on lab & what's expected

- Lab A, Topic8-RAG-Simple: cosine retrieval over a few sentences, cross-encoder re-ranking, generation, then vector memory
- Lab B, Topic8-RAG-with-Reranking-and-PDFs: chunk real PDFs, retrieve by cosine, re-rank the top-N’, generate a cited answer
- No vector DB: brute-force cosine is fine at this scale, and it keeps every step visible
- The code is complete; you change the embedding model, the chunking, the question, N’, the grounding rule and k
- Stretch: swap in a stronger embedding model or a vector store; wrap the retriever as a tool for your Topic 7 agent